

1. Introduction to Python

→ What is Python?

- Python is a high-level, interpreted, general-purpose programming language. It is easy to learn and read, making it perfect for beginners and powerful enough for professionals.

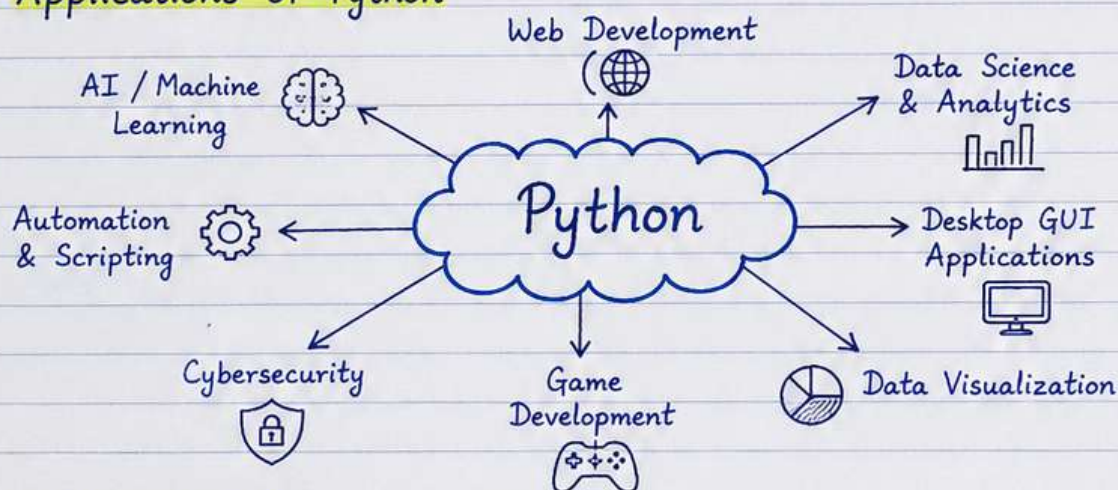
→ Why Python?

- Simple and easy syntax (like English)
- High-level language
- Dynamically typed
- Interpreted language
- Cross-platform (Windows, Mac, Linux)
- Large standard library
- Huge community support

→ Features of Python

- | | |
|--|---|
| <ul style="list-style-type: none">• Easy to learn• Readable and clean syntax• Open source• Portable | <ul style="list-style-type: none">• Extensible• Integrated• Automatic memory management• Object-oriented support |
|--|---|

→ Applications of Python



→ History (brief)

Python was created by Guido van Rossum and first released in 1991. It was named after the TV show "Monty Python's Flying Circus".

→ Your First Python Program

```
# This program prints Hello World
print("Hello World!")
```

Run this program and see the magic of Python 😊

Python is not just a language, it's a superpower! 🦄



2. Variables & Data Types

→ What is a Variable?

A variable is a container that stores data values in a program.

Memory Tip

Think of a variable as a box with a name and a value.



→ Rules for Naming Variables

- Can contain letters (a-z, A-Z), digits (0-9) and underscore (_)
- Must start with a letter or underscore
- Cannot start with a digit
- Case-sensitive (name, Name and NAME are different)
- Cannot use Python keywords (like if, else, for, class, etc.)

→ Built-in Data Types

Data Type	Description	Example	type() Output
int	Integer numbers	10, -25, 1000	<class 'int'>
float	Decimal numbers	10.5, -3.14, 0.001	<class 'float'>
str	Text / String	"Hello", 'Python'	<class 'str'>
bool	Boolean (True/False)	True, False	<class 'bool'>
NoneType	Special type (no value)	None	<class 'NoneType'>

→ Example (Program)

```
name = "Abhi"
age  = 22
height = 5.9
is_student = True
marks = None
```

```
# String
# Integer
→ # Float
# Boolean
# NoneType
```

→ Using type() Function

We can use type() to find the data type of any value.

```
type(name)      # <class 'str'>
type(age)       # <class 'int'>
type(height)    # <class 'float'>
type(is_student) # <class 'bool'>
type(marks)     # <class 'NoneType'>
```



Python is dynamically typed, meaning we don't need to declare the type of a variable.



Easy Right? 😊



3. Operators

Operators are used to perform operations on variables and values.

1. Arithmetic Operators

+	-	*	/	//	%	**
Addition	Sub	Mul	Div	Floor Div	Mod	Exponent

Example: $7/2 = 3.5$, $7//2 = 3$, $7\%2 = 1$
 $2**3 = 8$

2. Comparison Operators

==	!=	>	<	>=	<=
Equal	Not Equal	Greater	Less	Greater or Equal	Less or Equal

3. Assignment Operators

=	+=	-=	*=	/=	//=	%=	**=
Assign	Add	Sub	Mul	Div	Floor Div	Mod	Exponent
Assign	Assign	Assign	Assign	Assign			

Example: $a = 5$ $a += 3$ ($a = a + 3$)

4. Logical Operators

and	or	not
Returns True if both are True	Returns True if any one is True	Reverses the result

5. Membership Operators

in	not in
Returns True if value is present in the sequence	Returns True if value is not present in the sequence

6. Identity Operators

is	is not
Returns True if both variables point to the same object	Returns True if both variables do not point to the same object

7. Bitwise Operators (used with integers)

&		^	~	<<	>>
AND	OR	XOR	NOT	Left Shift	Right Shift

★ Interview Tip

Difference between == and is

== → checks value equality
(contents are same or not)

is → checks identity
(both point to same object in memory)

Example:

$a = [1, 2, 3]$

$b = [1, 2, 3]$

$a == b$ → True

$a is b$ → False

Quick Examples

1) Arithmetic

$a = 10$, $b = 3$

$a + b = 13$

$a // b = 3$

$a \% b = 1$

$a ** b = 1000$

2) Comparison

$5 > 3$ → True

$5 == 5$ → True

$5 != 3$ → True

3) Logical

True and False → False

True or False → True

not True → False

4) Membership

'a' in 'python' → True

'z' not in 'python' → True

5) Identity

$x = 10$

$y = 10$

$x is y$ → True



Operators are the building blocks of any program. Master them!



Next Page Roadmap (Page 4/15)

- Input in Python
- if Statement
- if...else
- if...elif...else
- Nested if
- Real-life Example

Let's learn decisions!



4. Input & Conditional Statements

1. Input in Python

- `input()` function is used to take input from the user.
- By default, `input()` returns data as string.

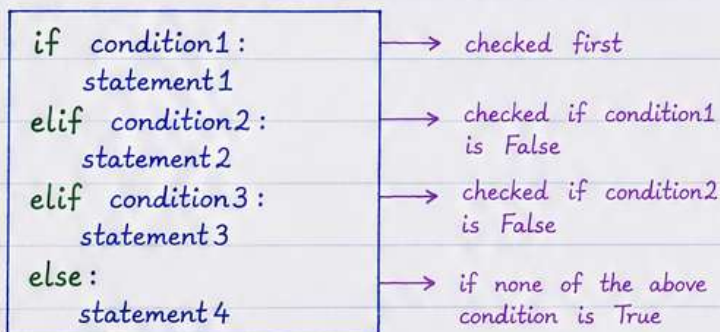
Syntax : `name = input("Enter something:")`

Example :

```
name = input("Enter your name:")
age = int(input("Enter your age:"))
# int() is used to convert string to integer
```

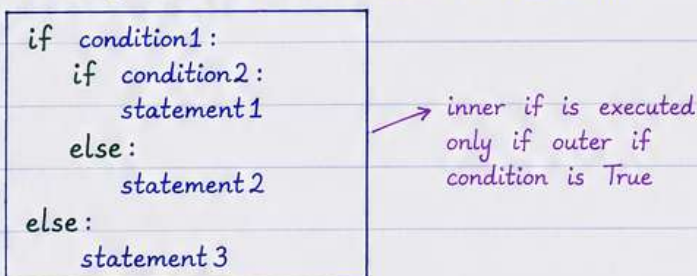
3. if, elif, else

- **if** → executes block if condition is True
- **elif** → checks another condition if previous was False
- **else** → executes if all conditions are False



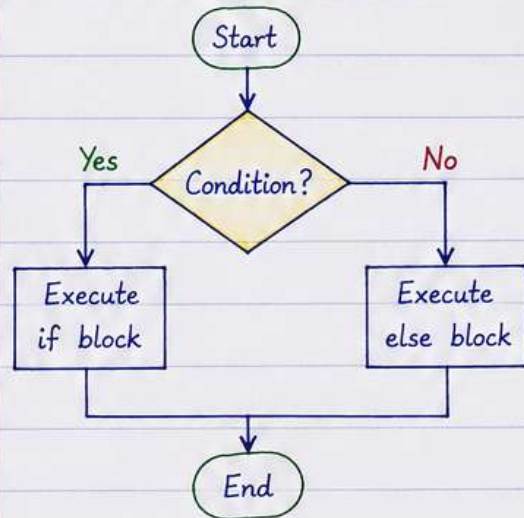
5. Nested if

- An if statement inside another if statement.



Always think: What should happen if condition is True?
And what if it is False?

2. if...else Flowchart



4. Real Example

Program to check vote eligibility based on age.

```
age = int(input("Enter your age:"))
if age >= 18:
    print("You are eligible to vote.")
elif age > 0:
    print("You are not eligible yet.")
else:
    print("Invalid age entered.")
```

Notes

- ★ In Python, indentation is very important.
- ★ Use 4 spaces for each block.
- ★ Any non-zero value is True in Python.
- ★ 0, False, None, "", [], {} are considered False.



Next Page Roadmap (Page 5/15)



- for loop
- while loop
- range() function
- break statement
- continue statement
- pass statement
- Nested loops example

Loops make your code powerful!





5. Loops in Python

Loops are used to repeat a block of code multiple times.

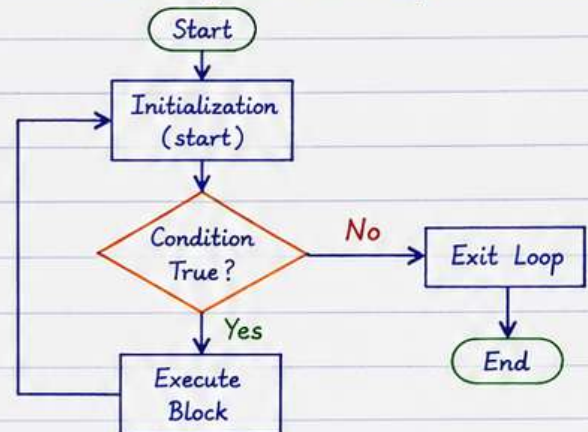
1. for Loop

- Used to iterate over a sequence (list, tuple, string, range, etc.)
- Known number of iterations.

```
for i in range(5):
    print(i)
```

Output : 0 1 2 3 4

Flow Diagram (for Loop)



2. while Loop

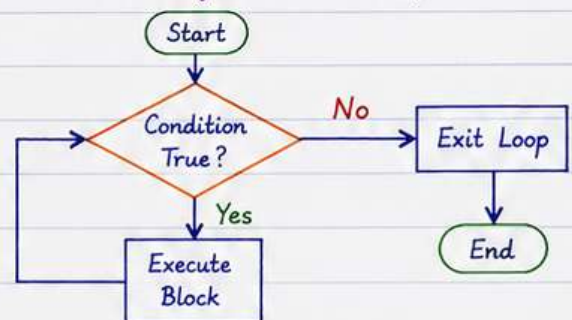
- Repeats a block while the condition is True.
- Used when number of iterations is not known.

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

Output :

1
2
3
4
5

Flow Diagram (while Loop)



3. Loop Control Statements

- ★ **break** : Terminates the loop completely.
- ★ **continue** : Skips the current iteration and continues with next.
- ★ **pass** : Does nothing. Used as a placeholder for future code.
- ★ **else** : Executes when loop completes normally (no break encountered).

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

Output : 0 1 2

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

Output : 0 1 3 4

```
for i in range(3):
    pass # no error
```

```
for i in range(3):
    print(i)
else:
    print("Loop Finished")
```

Output :
0 1 2
Loop Finished

4. range() Function

- Generates a sequence of numbers.
- Syntax : range(start, stop, step)
- start is inclusive, stop is exclusive.

range()	Output
range(5)	0 1 2 3 4
range(1, 6)	1 2 3 4 5
range(2, 10, 2)	2 4 6 8
range(10, 0, -2)	10 8 6 4 2

Next Page Roadmap (Page 6/15)

• Functions in Python

• Types of Functions

• Parameters
& Arguments

Lambda
Functions

Functions help
you write better,
cleaner code!
😊



6. Functions in Python



DRY Principle
Don't Repeat
Yourself

Functions are blocks of code that perform a specific task. They help in code reusability and make programs organized.

1. Defining a Function

```
def function_name(parameters):
    # body of the function
    return value
```

Example:

```
def add(a, b):          # parameters
    sum = a + b         # statement
    return sum          # return statement

result = add(10, 20)
print(result)           # Output: 30
```

2. Function Call

Functions are executed when they are called.

```
add(5, 7) # function call
```

Note:

A function is not executed until it is called. 😊

3. Parameters and Arguments

- **Parameters:** Variables listed in function definition.
- **Arguments:** Values passed to the function when calling it.

Example:

```
def greet(name):
    print("Hello", name)
```

parameter argument

```
greet("Abhi") # function call
```

4. Return Statement

- ★ The return statement sends a result back to the caller.
- ★ A function can return only one value.

Example:

```
def square(x):
    return x * x
```

```
num = square(6)
print(num) # Output: 36
```

5. Default Arguments

- ★ We can provide default value to parameters.

Example:

```
def greet(name, msg="Hello"):
    print(msg, name)
```

```
greet("Abhi")           # Hello Abhi
greet("Abhi", "Hi")     # Hi Abhi
```

6. Keyword Arguments

- ★ We can pass arguments using parameter name.

Example:

```
def info(name, age):
    print(name, age)
```

```
info(age=22, name="Abhi") # Abhi 22
```

7. Lambda Function (Anonymous Function)

- ★ Small one-line functions using lambda keyword.

Syntax:

```
lambda arguments: expression
```

Example:

```
square = lambda x: x * x
print(square(5)) # Output: 25
```

8. Multiple Return Values

- ★ A function can return more than one value as a tuple.

Example:

```
def operations(a, b):
    return a + b, a - b, a * b
```

```
x, y, z = operations(5, 3)
print(x, y, z) # 8 2 15
```



Functions make your code modular, reusable and easy to maintain.
Master functions, master Python! 😊

Next Page Roadmap
(Page 7/15)

- Strings in Python
- Indexing, Slicing, Methods, Formatting



7. Strings in Python

Strings are immutable sequence of characters enclosed in ' ' or " " 😊

1. What is String?

- String is a sequence of characters.
- In Python, we can use single quotes ('), double quotes (" ") or triple quotes (' ' ' ' ' ').
- Strings are immutable (cannot be changed).

Examples :

```
s1 = 'Hello'
s2 = "Python"
s3 = '''Multi
Line
String'''
```

2. Indexing in String

	s = "HELLO"				
Index :	0	1	2	3	4
Char :	H	E	L	L	O
Neg. Index :	-5	-4	-3	-2	-1

- Index starts from 0.
- Negative index starts from -1 (right to left).
- Access using s[index]

Example :

```
s = "HELLO"
print(s[0])    # H
print(s[4])    # O
print(s[-1])   # O
print(s[-5])   # H
```

3. Slicing in String

- Extract part of string using s[start : stop : step]

s =	P	Y	T	H	O	N
	0	1	2	3	4	5
	-6	-5	-4	-3	-2	-1

Example :

```
s = "PYTHON"
s[0:6] → PYTHON    # full string
s[0:3] → PYT        # start to 3-1
s[2:6] → THON       # 2 to end
s[1:5:2] → YH        # step = 2
s[::-1] → NOHTYP    # reverse string
```

4. Common String Methods

Method	Description	Example	Output
len(s)	Length of string	len("Hello")	5
s.upper()	Convert to upper case	"hello".upper()	"HELLO"
s.lower()	Convert to lower case	"HELLO".lower()	"hello"
s.capitalize()	First char upper, rest lower	"python".capitalize()	"Python"
s.title()	First char of each word upper	"python is fun".title()	"Python Is Fun"
s.strip()	Remove leading & trailing spaces	" abhi ".strip()	"abhi"
s.replace(a,b)	Replace a with b	"one two one".replace("one","1")	"1 two 1"
s.split()	Split string into list	"a,b,c".split(",")	["a", "b", "c"]
"sep".join(list)	Join list into string.	"-".join(["a", "b", "c"])	"a-b-c"

5. String Formatting

- Old style


```
name = "Abhi"
age = 22
print("My name is %s and age is %d" % (name, age))
```
- f-string (Recommended)


```
print(f"My name is {name} and age is {age}")
```
- format()


```
print("My name is {} and age is {}".format(name, age))
```

6. Escape Characters

```
\n : New line
\t : Tab
\' : Single quote
\" : Double quote
\\ : Backslash
\r : Carriage return
\b : Backspace
\f : Form feed
```

Example :

```
print("Hello\nPython")
↓
Hello
Python
```



Strings are Everywhere!

Master them and your Python journey becomes smoother. 😊

Next Page Roadmap (Page 8/15)

- Lists, Tuples and Sets
- Mutable vs Immutable
- Comparison Table



8. Collections in Python

Python has four main collection data types:
List, Tuple, Set and Dictionary.

Collections are used to store multiple items in a single variable. 😊

Comparison Table

Feature	List	Tuple	Set	Dictionary
Syntax	[]	()	{ }	{ key : value }
Ordered	Yes	Yes	No	Yes (Python 3.7+)
Mutable	Yes	No	No* (but items are unique)	Yes
Allows Duplicates	Yes	Yes	No	Keys - No Values - Yes
Indexing / Slicing	Yes	Yes	No	Keys - No Values - Yes
Heterogeneous Data	Yes	Yes	Yes	Yes
Use Case	Store ordered collection of items	Store fixed collection (can't change)	Store unique items (Unordered)	Store key-value pairs (mapping)

1. List (Mutable, Ordered)

- Defined using square brackets [].
- Can contain duplicate values.

Example :

```
fruits = ["apple", "banana", "mango", "apple"]
```

Common Methods :

- append() → add item at end
- insert(i, x) → insert at index i
- remove(x) → remove first occurrence
- pop(i) → remove & return item at i
- index(x) → return index of first occurrence
- count(x) → count occurrence
- sort() → sort the list
- reverse() → reverse the list

List Indexing

fruits				
A	B	C	D	
0	1	2	3	
-4	-3	-2	-1	

2. Tuple (Immutable, Ordered)

- Defined using parentheses ().
- Once created, cannot be changed.

Example :

```
days = ("Mon", "Tue", "Wed", "Thu")
```

Common Methods :

- count(x) → count occurrence
- index(x) → return index

Tuples are faster than lists. 😊

3. Set (Immutable, Unordered)

- Defined using curly braces { }.
- Stores only unique items.

Example :

```
nums = {1, 2, 3, 3, 4, 5} → {1, 2, 3, 4, 5}
```

Common Methods :

- add(x) → add item
- remove(x) → remove item (error if not present)
- discard(x) → remove item (no error if not present)
- union(s) → combine two sets
- intersection(s) → common items
- difference(s) → items in first set not in second

Venn Diagram



$A \cup B \rightarrow \text{union}$
 $A \cap B \rightarrow \text{intersection}$
 $A - B \rightarrow \text{difference}$

4. Dictionary (Mutable, Ordered)

- Stores data in key : value pairs.
- Keys must be unique and immutable (string, number, tuple).

Example :

```
student = {"name": "Abhi", "age": 22, "city": "Pune"}
```

Common Methods :

- keys() → return all keys
- values() → return all values
- items() → return key-value pairs
- get(key) → return value for key
- update(d) → update dictionary with d
- pop(key) → remove key and return value
- clear() → remove all items

Dictionary Structure

Key	Value
name	Abhi
age	22
city	Pune



Choose the right collection for the right job!
Understand the differences to write efficient Python code. 😊

Next Page Roadmap (Page 9/15)

- Dictionary in Python
- Create, Access, Update, Delete
- Dictionary Methods

9. Dictionary in Python

Dictionary stores data in key : value pairs. 😊

1. What is Dictionary?

- Dictionary is a collection of key : value pairs.
- Keys must be unique and immutable (string, number, tuple).
- Values can be of any data type.
- Dictionaries are mutable.

Example :

```
student = {
    "name" : "Abhi",
    "age" : 22,
    "city" : "Pune"
}
```

Key : Value Structure

Key	Value
name	Abhi
age	22
city	Pune

2. Creating Dictionary

- Using curly braces {}.
- Using dict() constructor

```
d1 = {}
d2 = {"a": 1, "b": 2}
d3 = dict(a = 1, b = 2)
d4 = dict([("a", 1), ("b", 2)])
```

3. Accessing Elements

- Using key
- Using get() method (safe way)

```
d = {"name": "Abhi", "age": 22, "city": "Pune"}
print(d["name"]) # Abhi
print(d.get("age")) # 22
print(d.get("country", "Not Found")) # Not Found
```

4. Updating Dictionary

- Update existing key
- Add new key

```
d["age"] = 23 # update
d["course"] = "Python" # add new
print(d)
```

5. Deleting Elements

- del → deletes key-value pair
- pop() → removes key and returns value
- popitem() → removes last inserted item
- clear() → removes all items

```
del d["city"]
age = d.pop("age")
item = d.popitem()
d.clear()
```

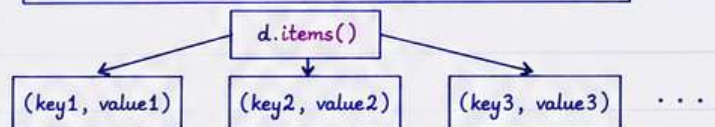
7. Dictionary Methods

```
keys() → returns all keys
values() → returns all values
items() → returns key-value pairs
get(key) → returns value for key
update(d) → updates dictionary with d
pop(key) → removes key and returns value
popitem() → removes last inserted item
clear() → removes all items
copy() → returns a copy of dictionary
```

6. Looping in Dictionary

- We can iterate using keys(), values() or items()

```
for k in d.keys(): print(k)
for v in d.values(): print(v)
for k, v in d.items(): print(k, "→", v)
```



8. Important Notes

- ★ Keys must be unique.
- ★ Dictionary is unordered (before Python 3.7).
- ★ From Python 3.7+, dictionaries maintain insertion order.
- ★ Access by key is very fast → O(1) average time.
- ★ Useful for quick lookups and mapping data. 😊

Example Programs

1. Count Frequency of Characters

```
s = "abacaba"
freq = {}
for ch in s:
    freq[ch] = freq.get(ch, 0) + 1
print(freq) # {"a": 4, "b": 2, "c": 1}
```

2. Student Marks

```
marks = {"Math": 95, "Eng": 88, "Sci": 90}
total = sum(marks.values())
avg = total / len(marks)
print("Average =", avg)
```

3. Invert a Dictionary

```
d = {"a": 1, "b": 2, "c": 3}
inv = {v: k for k, v in d.items()}
print(inv) # {1: 'a', 2: 'b', 3: 'c'}
```



Dictionary is your best friend for real-world data mapping and quick lookups!
Practice more to become a Python Pro! 🧐 😊

Next Page Roadmap
(Page 10/15)

• File Handling in Python



10. File Handling in Python

Files help us store data permanently. 😊

1. Opening a File

- We use `open()` function to open a file.
- Syntax: `open(filename, mode)`

Common Modes :

Mode	Mode	File must exist
'r'	Read (default)	file must exist
'w'	Write	creates new file or truncates existing
'a'	Append	adds data at end
'x'	Create	creates file, fails if exists
'b'	Binary mode	used with r, w, a
't'	Text mode (default)	used with r, w, a

2. Reading from a File

- `read()` → reads entire file
- `readline()` → reads one line
- `readlines()` → reads all lines into a list

Example :

```
f = open("hello.txt", "r")
content = f.read() # entire file
print(content)
f.close()
```



★ Don't forget to close the file using `close()` method! 😊

3. Writing to a File

- `write()` → writes a string to file
- `writelines()` → writes a list of lines

Example :

```
f = open("hello.txt", "w")
f.write("Hello Python!\n")
f.write("File handling is easy.")
f.close()
```



4. Appending to a File

- Adds new data at the end without deleting existing data.

Example :

```
f = open("hello.txt", "a")
f.write("\nThis line is appended.")
f.close()
```



5. Reading Lines

- `readline()` reads one line at a time.
- `readlines()` reads all lines and returns list.

Example :

```
f = open("hello.txt", "r")
line1 = f.readline()
line2 = f.readline()
all_lines = f.readlines()
f.close()
```

File Content	
Line 1	} <code>readline()</code>
Line 2	
Line 3	} <code>readlines()</code>
...	

6. Using with Statement (Best Practice)

- Automatically closes the file.
- Cleaner and safer.

Example :

```
with open("hello.txt", "r") as f:
    content = f.read()
    print(content)
```

File closes itself after block ends. 😊

7. File Methods Summary

Method	Purpose	Used For
<code>read()</code>	Read entire file	Small files
<code>readline()</code>	Read one line	Line by line reading
<code>readlines()</code>	Read all lines into list	Process all lines
<code>write()</code>	Write string to file	Overwrite (with 'w')
<code>writelines()</code>	Write list of lines	Write multiple lines
<code>close()</code>	Close the file	Free system resources

Example Program

```
# Create, write and read file
with open("demo.txt", "w") as f:
    f.write("Python is awesome!\n")
    f.write("File handling rocks!")

# Read the file
with open("demo.txt", "r") as f:
    for line in f:
        print(line.strip())
```



★ Practice file handling daily – it is essential for working with real data!
Learn today, build tomorrow. 😊

Next Page Roadmap (Page 11/15)

- Exception Handling
- Try, Except, Else, Finally
- Common Exceptions



11. Exception Handling in Python

Exceptions help handle errors gracefully without crashing! 😊

1. What is Exception ?

- An exception is an error that occurs during the execution of a program.
- Python provides a way to handle these errors so the program doesn't crash.

2. Basic Syntax

```
try:
    # code that might raise exception
except ExceptionType:
    # handle the exception
else:
    # runs if no exception occurs
finally:
    # always runs
```

3. Simple Example

```
try:
    num = int(input("Enter a number:"))
    result = 10 / num
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print("Result = ", result)
finally:
    print("Execution completed.")
```

finally block runs always! 😊

5. Multiple Except Blocks

```
try:
    a = int(input("Enter numerator:"))
    b = int(input("Enter denominator:"))
    print("Result = ", a / b)
except ZeroDivisionError:
    print("Denominator cannot be zero!")
except ValueError:
    print("Please enter valid integers!")
except Exception as e:
    print("Some other error occurred:", e)
```

7. Raising Exceptions

- We can raise exceptions manually using raise.
- Useful to enforce conditions in our code.

```
def check_age(age):
    if age < 18:
        raise ValueError("Age must be 18 or above!")
    print("Access granted")

check_age(16)    # Raises ValueError
```

4. Else and Finally in Action

- else runs only if no exception occurs.
- finally runs always, exception or not.

```
try:
    x = int(input("Enter a number:"))
    print("Number is ", x)
except ValueError:
    print("Invalid input!")
else:
    print("No exception occurred.")
finally:
    print("This will always run.")
```

else runs only when try block is successful.

→ finally runs always! 😊

6. Common Built-in Exceptions

Exception	Description
ZeroDivisionError	Raised when division by zero occurs
ValueError	Raised when a function gets an argument of correct type but inappropriate value
TypeError	Raised when operation or function is applied to inappropriate type
FileNotFoundError	Raised when a file is not found
IndexError	Raised when index is out of range
KeyError	Raised when key is not found in dictionary

8. Custom Exception

- We can create our own exception by creating a new class.
- It should be inherited from Exception class.

```
class InvalidAgeError(Exception):
    pass

def validate_age(age):
    if age < 0 or age > 120:
        raise InvalidAgeError("Invalid age!")

try:
    validate_age(200)
except InvalidAgeError as e:
    print("Custom Exception: ", e)
```

Custom exceptions make code more meaningful! 😊



Handle exceptions, don't ignore them.

Your program will become more reliable and user friendly! 😊

Next Page Roadmap (Page 12/15)

- Modules in Python
- Import, from ... import
- Built-in Modules
- Creating Your Own Module



12. Modules in Python

Modules help us organize code into reusable files! 😊

1. What is a Module?

- A module is a file containing Python code (functions, classes, variables).
- We can import and reuse code from other modules.
- Every Python file is a module.

mymath.py

```
def add(a, b):
    return a + b

def sub(a, b):
    return a - b
```

We can import this module in another file.

main.py

```
import mymath
result = mymath.add(5, 3)
print(result)
```

2. Creating a Module

- Create a Python file with .py extension.
- Write functions, classes, variables in it.

Example: mymath.py

```
def add(a, b):
    return a + b

def mul(a, b):
    return a * b

PI = 3.14159
```

This file is now a module named mymath

3. Importing Modules

- We use import statement.
- Two ways to import:

(a) Import the whole module

```
import mymath
print(mymath.add(2, 3))
print(mymath.PI)
```

(b) Import specific items

```
from mymath import add, mul
print(add(2, 3))
print(mul(2, 3))
```

★ Using specific import makes code shorter and easier to read. 😊

4. Using 'as' Keyword

- We can use as to give alias to module.

```
import mymath as mm
print(mm.add(4, 5))
print(mm.PI)
```

mm is an alias for mymath

6. Example - Using Built-in Module

- Using math module

```
import math
print(math.sqrt(16)) # 4.0
print(math.pi)      # 3.141592653589793
```



5. Built-in Modules

- Python has many built-in modules.
- Some commonly used modules:

Module	Purpose
math	Mathematical functions
random	Generate random numbers
datetime	Work with dates and times
os	Interact with operating system
sys	System-specific parameters

No need to create these modules, they are ready to use! 😊

7. The __name__ Variable

- Every Python module has a special variable `__name__`.
- When a module is run directly, `__name__` is set to `"__main__"`.
- Otherwise, it is set to the module name.

Example: mymodule.py

```
def greet():
    print("Hello from module!")

if __name__ == "__main__":
    greet() # Runs only if file is executed directly
```

Good for testing code in a module.

8. Benefits of Modules

- ☑ Improves code readability and organization.
- ☑ Encourages code reusability.
- ☑ Makes large programs easier to manage.
- ☑ Promotes teamwork (multiple developers can work on different modules).

Tips

- ★ Keep module names short and meaningful.
- Avoid naming your module same as built-in modules.
- Use relative paths or packages for large projects. 😊



Modules are building blocks of Python programs.

Master them to write clean, reusable and professional code! 😊

Next Page Roadmap (Page 13/15)

- Packages in Python
- Creating and Using Packages
- `__init__.py` and Package Structure



13. Packages in Python

Packages are collections of modules in a directory hierarchy. 😊

1. What is a Package?

- A package is a way of organizing related modules in a directory.
- It contains a special file called `__init__.py`
- Packages help in code reusability and scalability.

Example Package Structure

```
mypackage/
├── __init__.py
├── math_utils.py
├── string_utils.py
└── main.py
```

mypackage is a package. It contains modules.

3. Importing from a Package

- We can import modules or specific items from a package.

(a) Import Module

```
from mypackage import math_utils
result = math_utils.add(5, 3)
print(result)
```

(b) Import Specific Item

```
from mypackage.math_utils
import add
result = add(5, 3)
print(result)
```

★ Always use dot (.) operator to access modules and functions. 😊

5. Relative vs Absolute Import

Absolute Import	Relative Import
from mypackage import math_utils	from . import math_utils
✓ Starts from top-level package.	✓ Starts from current package.
✓ Works clearly and is preferred.	✓ Used inside packages.

Use absolute imports in main programs. Relative imports are useful inside packages. 😊

7. Installing External Packages

- External packages are not included in Python.
- Install using pip.

```
pip install package_name          # install latest version
pip install package_name==1.2.0   # specific version
```

Example :

```
pip install requests
pip install numpy==1.24.0
```



Install from Python Package Index (PyPI)



Organize your code with packages – it makes your projects clean, scalable and easy to maintain! 😊

2. Creating a Package

- Create a folder with the package name.
- Inside it, add a file named `__init__.py` (can be empty).
- Add your module files in the same folder.

Example :

```
mkdir mypackage
cd mypackage
touch __init__.py    # create empty file
touch math_utils.py
touch string_utils.py
```



4. The __init__.py File

- This file is executed when the package is imported.
- We can use it to initialize the package.
- It can also contain package-level variables or import statements.

Example : __init__.py

```
print("Initializing mypackage...")
__version__ = "1.0.0"
from .math_utils import add
```

Executed when package is imported.



6. Built-in / Standard Packages

- Python comes with many built-in packages in the standard library.
- Some useful ones :

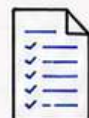
```
★ math      - Mathematical functions
★ random    - Generate random numbers
★ datetime  - Work with date & time
★ os        - Operating system interaction
★ sys       - System-specific parameters
★ json      - Work with JSON data
```

No need to install these, they are ready to use! 😊

8. Listing Installed Packages

- See all installed packages using pip.

```
pip list    # list installed packages
pip freeze  # list with versions
```



Uninstall a package :

```
pip uninstall package_name
```

Next Page Roadmap (Page 14/15)

- Virtual Environments
- Requirements File
- Best Practices for Packages



14. Virtual Environments in Python

Virtual environments keep projects independent and dependency conflicts away! 😊

1. What is a Virtual Environment?

- A virtual environment (venv) is an isolated Python environment.
- It has its own Python interpreter and packages.
- Helps avoid dependency conflicts between projects.
- Keeps global Python installation clean.

Think of it as a separate workspace for your project.



2. Why Use Virtual Environments?

- ★ Different projects may need different versions of the same package.
- ★ Prevents "it works on my machine" problems.
- ★ Easy to share project requirements.
- ★ Keeps system Python clean & safe.
- ★ Good practice for professional development. 😊

3. Creating a Virtual Environment

- Python comes with built-in `venv` module.
- Create a new virtual environment using:

```
# Windows
python -m venv myenv

# macOS / Linux
python3 -m venv myenv
```

↖ This creates a folder "myenv" with its own Python and packages.

4. Activating the Virtual Environment

- Activate it before using.

Windows (Command Prompt)

```
myenv\Scripts\activate
```

macOS / Linux (Terminal)

```
source myenv/bin/activate
```

★ After activation, your prompt will look like:
(myenv) C:\Users\Abhi> (myenv) abhi@mac ~ \$

5. Deactivating the Environment

- When done, deactivate it:

```
deactivate
```

↖ Works on all platforms.



6. Installing Packages in the Environment

- All packages installed now will be inside this environment.
- Use `pip` as usual.

```
pip install numpy
pip install requests
```

↖ Installed only in this virtual environment. 😊

7. Viewing Installed Packages

- List all packages in the current environment.

```
pip list
```

↖ Shows all installed packages.

Or with details:

```
pip freeze
```

↖ Shows exact versions. Useful for sharing.

9. Installing from Requirements File

- Recreate the same environment on another machine.

```
pip install -r requirements.txt
```

↖ Installs all packages listed in the file.

8. Saving Dependencies

- Save installed packages to a requirements file.

```
pip freeze > requirements.txt
```

↖ Creates a file with all packages and versions.

10. Best Practices

- ☑ Always use virtual environments for projects.
- ☑ Don't install packages globally.
- ☑ Add `.venv/` or `myenv/` to `.gitignore`.
- ☑ Share `requirements.txt`, not the environment folder.
- ☑ Keep dependencies updated regularly. 😊



★ Virtual environments = Clean Code + Happy Projects!
Small step now, big save later! 🐍 😊

Next Page Roadmap (Page 15/15)

- Useful Python Libraries
- Python Tips & Tricks
- Summary & Final Notes



15. Summary & Final Notes

You've made it!
Keep practicing and
keep building. 😊

1. File Handling - Key Points

- Python makes file handling simple and powerful.
- Always close files (use with open() for safety).
- Choose the correct mode for the job.
- Handle exceptions for robust code.

★ Golden Rule

Always close the file.

A file left open can cause problems! 😊

2. Exception Handling - Key Points

- Exceptions help your program handle errors gracefully.
- Use try - except to catch and handle errors.
- Use else and finally to control flow.
- Raise custom exceptions for better validation.



Remember

A good program doesn't avoid errors,
it handles them smartly. 😊

3. Modules - Key Points

- Modules keep code organized and reusable.
- Import only what you need.
- Use built-in modules before writing your own.
- Follow clean structure and meaningful names.

Common Built-in Modules

math - Math functions	sys - System parameters
random - Random numbers	json - JSON handling
datetime - Date & time	re - Regular expressions
os - OS interactions	collections - Specialized containers

Use these
modules
to save
time and
write less
code! 😊

4. Virtual Environments - Key Points

- Isolates dependencies for each project.
- Prevents "it works on my machine" issues.
- Keeps global Python installation clean.
- Easy to create, activate and share.

Basic Workflow



Create
venv



Activate
venv



Install
packages



Work on
your project

5. Quick Reference - Commands

Task	Command / Code
Open a file	open("file.txt", "r")
Read all content	f.read()
Write to file	f.write("text")
Append to file	open("file.txt", "a")
Handle exception	try: ... except Exception as e: ...
Import module	import module_name
Import specific item	from module import item
Create venv (Windows)	python -m venv myenv
Activate venv (Win)	myenv\Scripts\activate
Activate venv (macOS/Linux)	source myenv/bin/activate
Install package	pip install package_name

6. Best Practices Checklist

- ✓ Always use with open() for file handling.
- ✓ Handle possible exceptions in your code.
- ✓ Keep functions and modules small and focused.
- ✓ Use meaningful variable, function and module names.
- ✓ Use virtual environments for every project.
- ✓ Install only the packages you need.
- ✓ Keep dependencies updated.
- ✓ Write clean, readable and commented code.
- ✓ Test your code with different scenarios.
- ✓ Keep learning and building projects! 😊



★ Congratulations!

You have completed the Python Handbook.

Practice daily, build projects and apply what you learn.

The more you code, the better you become! 📖 😊

What's Next?

- Build real-world projects
- Explore advanced Python topics
- Contribute to open source
- Never stop learning! 😊